

specs/

Spec-Driven Development

Harness Engineering for AI coding agents.

Josh Janzen

Data Scientist · SDS Bootcamp Alum

June 3, 2026

CORE Session
for SuperDataScience
AI Engineering Alumni



AI Engineer
(Losing Control /
Failed Human-in-the-Loop)

Vibe Coding
(The Act of Speculating
with Snapped Reins)

Harness Engineering
(Orchestration in Crisis / Saddle Slipping)

The Agent
(Rogue, Unbridled /
Large Muscular Horse)

V I B E - C O D I N G

The act of speculating with snapped reins.



AI Engineer

The System Director
(Human in the Loop)

The Agent
(Large Muscular Horse)

Spec-Driven Development
(The Act of Pointing and Directing Reins)

Harness Engineering
(Orchestration)

SPEC-DRIVEN DEVELOPMENT

The act of pointing and directing the reins.

Copyright © 2026 Josh Janzen

SDD begins with three questions

Why

are we building this?

Mission. What outcome justifies the work — for the user, the team, the business.

What

are we building?

Tech stack and scope. Languages, frameworks, services, schema, constraints.

How

will it be used?

Roadmap and usage. Who runs it, how often, what “done” looks like in their hands.

Answer these before you write a line of code — or let the agent write one.

Vibe-Coding vs. TDD vs. SDD

Vibe-Coding	TDD	SDD
Prompt → Code	Test → Code → Refactor	Spec → Plan → Code → Validate
SOURCE OF TRUTH	SOURCE OF TRUTH	SOURCE OF TRUTH
The prompt (ephemeral)	Failing tests	The constitution + spec
VALIDATES AGAINST	VALIDATES AGAINST	VALIDATES AGAINST
“Looks right”	Unit assertions	Acceptance criteria
STRENGTH	STRENGTH	STRENGTH
Speed for throwaway work	Local correctness, regression	Aligns AI to business intent
FAILURE MODE	FAILURE MODE	FAILURE MODE
Drift, hallucination, rework	Tests ≠ requirements	Specs go stale if not replanned

SDD is Harness Engineering



The practice

Write a constitution. Spec the feature. Plan the work. Implement against the plan. Validate.

You don't write the code — you steer the agent that does. Specs are the reins; the harness is the system around them. Focus shifts from typing code to designing contracts.

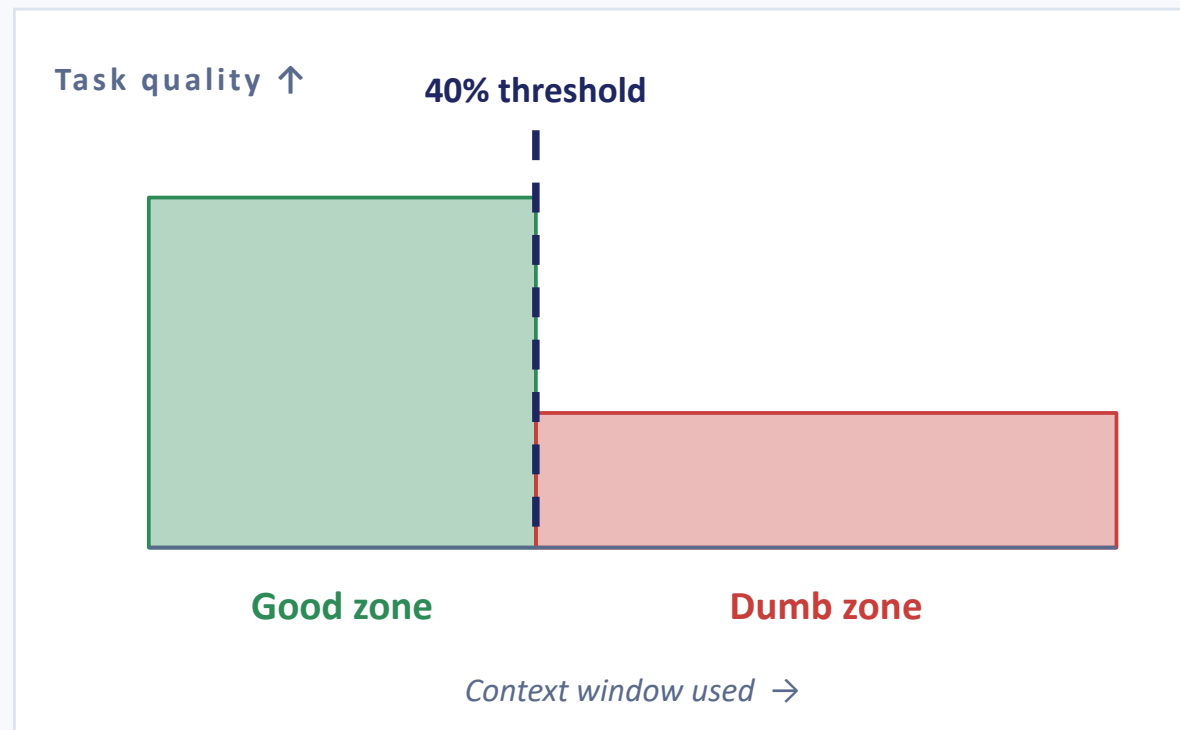
The shift

From writing code → directing systems

Before	Typing every line; agent assists
Now	Specifying intent; agent implements
Before	Drift accumulates across long sessions
Now	Each feature is a fresh, scoped context
Before	“It ran” is the definition of done
Now	Acceptance criteria are the definition

Avoid the “dumb zone”

Research shows agent task performance degrades sharply once context exceeds ~40% of the window. Bigger isn't better — focused is better.



Progressive disclosure

Show the agent only what it needs

- ✓ **Scoped specs** *Each feature lives in its own folder.*
- ✓ **Fresh contexts** */clear between phases keeps the window lean.*
- ✓ **Skills > prompts** *Reusable, narrow instructions per task.*
- ✓ **Pre-stated stack** *Tell the agent your tools up front, once.*

Repo entry points & the constitution

Repo-root entry files

AGENTS.md

Front door every agent reads — tech stack, commands, conventions

TODO.md

The inbox. Raw, unvetted ideas. Messy on purpose.

README.md

Anything else — voice transcripts, sketches, free-form context

The Constitution

Lives in specs/ — the source of truth



mission.md

Why — outcome, audience, stakes



tech-stack.md

What — frameworks, services, schema



roadmap.md

How — phased implementation order

THE WORKFLOW

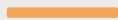
Six steps, on a loop

Required

1

Constitution

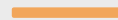
mission · tech-stack · roadmap



2

Feature Spec

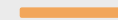
plan · requirements · validation



3

Implement

execute the plan on a branch



Stay Disciplined

4

Replan

update the constitution



5

Changelog

compress completed phases



6

Pre-flight

/clear and check before next



Each step is now packaged as a /sdd- skill (slash command) — demo will show them in action*

Constitution → Feature Specification

STEP 1 · CONSTITUTION

Greenfield vs Brownfield

Greenfield: interview for mission, audience, tech stack — then write the three files.

Brownfield: seed roadmap from existing TODO.md; interview for gaps.

Skills:

- /sdd-constitution-greenfield
- /sdd-constitution-brownfield

STEP 2 · FEATURE SPEC

specs/YYYY-MM-DD-feature-name/



plan.md

numbered task groups



requirements.md

scope, decisions, context



validation.md

how we know it's done

Skills:

- /sdd-roadmap-from-todo
- /sdd-feature-spec

Implement → Replan

STEP 3 · IMPLEMENT

Execute the plan

- 1 Hand the agent plan.md**
Implementation references the spec, not the conversation.
- 2 Review the commits**
Locally run or test. Sanity-check the work, not the prose.
- 3 Tweak or replan**
Small drift — edit. Big drift — revise the plan, redo.
- 4 Mark phase complete**
Update roadmap.md and commit.

Skill: /sdd-implement-feature

STEP 4 · REPLAN

Close the loop

Once a feature lands, the constitution may now be out of date. Open a replanning branch and ask the agent to update mission.md, tech-stack.md, and roadmap.md to reflect new reality.



Optional: write new tests against the updated constitution.

Skill: /sdd-replan

Changelog → Pre-flight

STEP 5 · CHANGELOG

Keep the roadmap clean

Move every [x] COMPLETE phase out of roadmap.md and into changelog.md. Compress to title, date, and one-sentence outcome — with a link to the spec dir.

```
roadmap.md
open + upcoming phases only
↓
changelog.md
compressed history
```

Skill: /sdd-changelog

STEP 6 · PRE-FLIGHT

Four questions before the next item

- ✓ Do I have unfinished work?
- ✓ Did I merge the last branch to main?
- ✓ Is the next roadmap item the right thing?
- ✓ Did I clear the agent's context?

Then: /clear

Skill: /sdd-preflight

The independent validator

Use a second agent session to verify the work — with deliberately incomplete context. Separation of concerns prevents plan-shaped reviews.

✓ Validator MAY read

- `mission.md`
- `tech-stack.md`
- `validation.md` (for this feature)
- all source code on the branch
- test files and test output

✗ Validator MUST NOT read

`plan.md`

Why

If the validator reads the plan, it checks whether code matches the plan. Its job is to check whether code matches the acceptance criteria.

Skill: `/sdd-independent-validator`

Built on multiple sources



How this came together

Synthesized, not invented

- **Conference talks** *AI Engineer, dev podcasts*
- **YouTube creators** *Practitioners running SDD in production*
- **Frameworks** *Spec-Kit, Agent OS, Kiro*
- **Online courses** *DeepLearning.AI × JetBrains*
- **Press · trend pieces** *Harness engineering coverage in 2026*

My contribution

Each step packaged as a slash command

```
skills/  
  /sdd-constitution-greenfield  
  /sdd-constitution-brownfield  
  /sdd-roadmap-from-todo  
  /sdd-feature-spec  
  /sdd-implement-feature  
  /sdd-independent-validator  
  /sdd-validation-todo  
  /sdd-replan  
  /sdd-changelog  
  /sdd-preflight
```

CREDIT WHERE DUE

References

COURSE

Spec-Driven Development with Coding Agents

DeepLearning.AI × JetBrains

deeplearning.ai

Talks & videos

No Vibes Allowed: Solving Hard Problems in Complex Codebases

Dex Horthy, HumanLayer · AI Engineer (origin of the “dumb zone”)

I Used GitHub's Spec Kit for 30 Days. Here's the Truth.

Gui Ferreira · @gui.ferreira

Spec-Driven Development in the Real World

Brian Casel · @briancasel

Horse images: generated by Google Gemini

Takeaways — then a live build

01 Start with three questions

Why? What? How will it be used?

02 Be a harness engineer

Steer the agent. Don't out-type it.

03 Stay out of the dumb zone

Specs are progressive disclosure.

04 Validator ≠ implementer

Independent context, honest reviews.



Next — 60-min live demo: building an app with SDD, end-to-end.

Materials: GitHub repo: <https://github.com/zen-apps/spec-driven-app>